

# AgentPProf: Semantic Profiling for AI Agent Traces

## Abstract

AI agents produce complex execution traces spanning prompts, LLM calls, tool invocations, GUI actions, and system effects. As deployments grow to hundreds of sessions, both human operators and automated analysis pipelines need aggregate views—not just per-execution debugging.

Existing observability tools organize events into fixed session/span hierarchies suited for single-run debugging but unable to fold events across sessions by semantic work category. CPU flame graphs solve the equivalent aggregation problem for programs, but they rely on deterministic function names. Agent prompts are natural language and cannot be folded directly.

We show that agent profiling reduces to two abstractions once intent recognition restores the deterministic-label precondition. An *operation* is a uniform weighted record for any agent activity; an *operation stack* is a field projection chosen at query time. Sessions and spans become optional fields, not fixed hierarchy nodes. AGENTPPROF implements this model with pluggable intent-recognition backends (regex rules, local LLM tagging, unsupervised clustering) and produces flame graphs, pprof profiles, and structured JSON consumable by both developers and analysis agents.

On 325 real agent sessions, semantic labels separate 90% of system-effect weight that flat views merge. On four public datasets (34,539 operations), operation-stack profiling reduces inspection work by 90% versus flat summaries and fragmentation by 45% versus fixed-session drilldown. Leave-dataset-out mapping achieves V-measure above 0.7 on 7/9 held-out datasets.

## 1 Introduction

AI agent systems produce heterogeneous execution traces spanning prompts, LLM calls, tool invocations, GUI actions, and system effects. As deployments scale to hundreds of sessions and tens of thousands of calls, both developers and automated analysis agents need to answer aggregate questions: which categories of work consume the most tokens, where do repeated failures concentrate, and how does cost distribute across semantic task families?

Existing observability tools—LangSmith [6], Langfuse [7], Phoenix [8], OpenTelemetry GenAI [4]—organize events into per-execution timelines and span trees, enabling single-run debugging. But no existing tool provides an aggregate view that folds heterogeneous events across sessions into a cost or frequency distribution grouped by semantic work category.

CPU profilers solved the equivalent problem for programs: flame graphs [1] fold millions of function-call samples into a single chart where width represents cost. The key enabler is *deterministic* function names that merge identical call stacks. Agent traces break this assumption—prompts are natural language, and two prompts expressing the same intent share no characters. We develop this gap and the design requirements it implies in §2.

Some existing tools take steps toward semantic grouping: Datadog LLM Observability clusters user messages by topic,<sup>1</sup> and Laminar extracts structured signals from traces.<sup>2</sup> These tools characterize *input distributions* (“30% of users ask code questions”) but not *resource attribution* (“code-review tasks consume 40% of the token budget”).

The central insight is that agent observability requires *profiling*: aggregate, cross-session cost attribution grouped by user intent—which we call *semantic* to distinguish from execution-structural groupings like sessions or spans. The obstacle to applying classical profiler techniques is the nondeterminism of natural-language identifiers.

This paper presents AGENTPPROF, a profiler that restores aggregate flame-graph analysis for agent traces. The profiler is built on two abstractions: an *operation* (a uniform weighted record for any agent activity) and an *operation stack* (a field projection chosen at query time). Sessions, spans, and trace identifiers are operation fields, not fixed hierarchy nodes. *Intent recognition*—mapping free-form prompts to stable labels through pluggable backends (regex rules, local LLM tagging, or unsupervised clustering)—restores the deterministic-label precondition that flame-graph folding requires.

This paper makes three contributions:

- (1) **Insight: agent profiling reduces to two abstractions.** We show that the aggregation gap in agent observability—fixed session/span hierarchies cannot fold by semantic category—is closed by combining uniform operation records with query-time stack projections, once intent recognition restores deterministic labels (§2–§3).
- (2) **System: AGENTPPROF.** We implement this model as a Rust CLI with pluggable intent recognition, four built-in views, and output formats (pprof, SVG, JSON) consumable by both developers and automated analysis agents (§4).
- (3) **Evaluation.** We show that semantic labels separate 90% of system-effect weight that unsemantic views merge (RQ1), that operation-stack profiling localizes hidden labeled problems with 90% less work than flat summaries (RQ2), and that mapping-derived labels agree with ground-truth annotations on 7/9 public datasets (RQ3) (§5).

## 2 Background and Motivation

### 2.1 How Existing Agent Trace Models Work

Current agent observability tools organize traces around fixed hierarchies. OpenTelemetry represents an execution as a *trace* containing a tree of *spans*, each with a name, duration, and key-value attributes. LangSmith and Langfuse group spans into *sessions* and display them as timelines or waterfall charts. Phoenix and OpenInference add GenAI-specific span types for LLM calls, tool invocations, retrieval steps, and agent reasoning.

These models share a structural property: the session or span tree is the skeleton of the data. Every event belongs to exactly one

<sup>1</sup>[https://docs.datadoghq.com/llm\\_observability/](https://docs.datadoghq.com/llm_observability/)

<sup>2</sup><https://docs.lmn.ai/signals/introduction>

span, and spans nest in a fixed parent-child hierarchy determined at execution time. This design is effective for debugging: a developer can locate a failing span, inspect its attributes, and trace its parent chain to understand the execution context.

## 2.2 Why Fixed Hierarchies Cannot Profile

Profiling requires a fundamentally different operation: *folding* events across executions by semantic category so that width represents aggregate cost. Fixed session/span hierarchies obstruct this in three ways.

First, sessions are boundaries, not categories: a single session may contain review, debugging, and refactoring phases, and splitting by session assigns all of them to the same bucket. Conversely, two sessions performing the same task produce separate buckets that cannot be merged without external logic. Second, span names describe mechanisms (LLMCall, ToolInvoke, ChatCompletion), not user intent (review, debug, test). Grouping by span name tells you how many LLM calls occurred, not which categories of work consumed the budget. Third, these limitations compound in practice: in a real development project, the system effect process: cargo; effect: test; status: ok appears 2,903 times across 325 sessions. A flat process summary shows a single counter. No existing trace tool can disaggregate these executions by the semantic intent that triggered them. We quantify this disaggregation in RQ1 (§5).

## 2.3 Why CPU Profiling Does Not Directly Apply

CPU profilers solve the equivalent aggregation problem for programs. A flame graph [1] represents a profile as a set of weighted *samples*, each attached to a *call stack* (an ordered list of function frames). Identical stacks merge, and width represents aggregate cost. The key enabler is that function names are *deterministic*: the same code path always produces the same frames.

Agent traces break this precondition. As noted in §1, prompts are natural language and lack deterministic identifiers. Tool calls, GUI actions, and process events are named heterogeneously across agent families. Raw event names cannot serve as flame-graph frames.

These observations yield three design requirements:

**R1: Uniform record type.** Agent traces mix prompts, LLM calls, tool invocations, GUI actions, process events, and benchmark labels. The profiler needs a single record type that can represent all of them without type-specific objects.

**R2: Query-time aggregation.** The same data must support multiple aggregation shapes—by task, phase, action, session, or dataset—without rebuilding the trace. Sessions and spans must be fields, not fixed hierarchy nodes.

**R3: Deterministic labels for folding.** Free-form prompts must be mapped to short, stable, repeatable labels before folding. This *intent recognition* step has no analogue in CPU profiling, where function names are already deterministic.

## 3 Design

The three requirements—a uniform record type (R1), query-time aggregation (R2), and deterministic labels for folding (R3)—drive the design. We introduce two abstractions—operations (§3.1) and operation stacks (§3.2)—that satisfy R1 and R2, and a pluggable intent-recognition mechanism (§3.3) that satisfies R3.

**Table 1: Built-in views. Each shares the same operation set but selects different activities, stack structures, and weights.**

| View    | $\varphi$ (select) | $w$ (width)  | Question             |
|---------|--------------------|--------------|----------------------|
| tokens  | LLM calls          | token count  | Budget distribution? |
| time    | All timed ops      | duration (s) | Time distribution?   |
| files   | File-effect ops    | event count  | Repo coverage?       |
| network | Net-effect ops     | event count  | External contacts?   |

### 3.1 Operations (R1: Uniform Record Type)

A profiler should not distinguish between prompts, tool calls, and process events at the object-model level. AGENTPPROF uses a single abstraction: an operation is a weighted record with string fields and additive measures.

Each operation carries string fields (project, agent, session, prompt\_tag, kind, model, path, domain, status, ...) and additive measures (token count, duration, occurrence count). A prompt, an LLM call, a tool invocation, a file read, a GUI click, and a process event are all operation records that differ only in which fields are populated.

### 3.2 Operation Stacks (R2: Query-Time Aggregation)

An operation stack is an ordered list of fields chosen at query time. Folding operations by that list produces the standard folded-stack representation:

```
CPU:  main;parse;tokenize          42
Agent: prompt:debug;call:llm;tool:bash 4200
```

Unlike CPU profiling, where the call stack is determined by execution, the operation stack is determined by the query. The same operations can be folded as a flat task summary, a fixed-session tree, a semantic phase/action hierarchy, or a dataset-native trajectory. Changing the fields changes the aggregation without changing the data.

This directly addresses the fixed-hierarchy problem from §2: sessions and spans are optional operation fields, not structural skeleton nodes. Including session in the stack produces a per-session drilldown (equivalent to a trace tree); excluding it produces a cross-session aggregate (which no existing trace tool provides). The same data supports both debugging and profiling views.

A view is a triple  $(\varphi, \sigma, w)$ : a predicate  $\varphi$  selects which operations participate, a stack function  $\sigma$  maps each operation to a frame sequence, and a weight function  $w$  assigns a nonnegative measure. The view formalism supports arbitrary user-defined views; four built-in defaults (Table 1) answer common questions:

### 3.3 Intent Recognition (R3: Deterministic Labels)

Operations and stacks handle heterogeneity and flexible aggregation, but folding still requires deterministic frame labels—the precondition that agent prompts violate (§2). AGENTPPROF solves this through *intent recognition*: mapping free-form prompts to short, stable labels before stack construction.

[Architecture: Agent trace files (Codex/Claude JSONL, benchmark JSONL) → Parser / Importer → Intent Recognition (regex / LLM / clustering) or Mapping Rules → Operations with fields → Stack query ( $\varphi, \sigma, w$ ) → Folded stacks → Flame graph / pprof / JSON]

**Figure 1: System architecture. Local agent histories enter through agent-session parsing; public datasets enter as operation JSONL. Both paths produce operations; the stack query and output are identical.**

The design uses a pluggable tagger framework with three back-ends:

*Regex rules with agent-assisted iteration.* The production default. Rules have the form `KIND:TAG=REGEX`, where `KIND` is prompt, `llm`, or `all`, `TAG` is a short lowercase label, and `REGEX` is a pattern. Rules are evaluated in order; the first match wins. The workflow is human-agent collaborative: run AGENTPPROF, observe the unmatched rate and sample prompts, add rules, re-run, and iterate until unmatched drops below 5%. This typically takes 5–10 rounds. The resulting rule set is deterministic, reproducible, and version-controllable.

*Local LLM tagger.* For exploration or complex multilingual prompts, a local model (e.g., a 3B-parameter GGUF via `llama.cpp`) generates a single-word label per prompt. Grammar-constrained decoding ensures output validity. Labels are cached, so re-runs with unchanged prompts incur zero LLM calls.

*Unsupervised clustering.* TF-IDF vectorization followed by K-Means discovers natural prompt groups without predefined rules, feeding rule authoring.

The one-word label design is deliberate. Flame-graph frames must be short because long summaries make charts unreadable. Labels serve navigation and aggregation, not correctness verdicts.

For structured traces—public benchmark datasets and trace formats where action types, tool names, and quality annotations are explicit fields—intent recognition is unnecessary. AGENTPPROF supports deterministic *mapping rules* that derive new operation fields from existing ones before stack construction. A *profile spec* records the complete analysis configuration—data sources, mappings, predicates, stack fields, and outputs—for deterministic replay. This dual path explains why AGENTPPROF handles both real agent sessions (which need intent recognition) and public benchmark traces (which need mapping rules): both paths produce operations with fields, and the stack query works identically on both.

## 4 Implementation

AGENTPPROF is an offline, post-hoc profiler implemented as a Rust CLI (Figure 1). It reads recorded traces, not live events. The `agent-session` crate reads Codex and Claude Code local JSONL history files and reconstructs prompts, LLM calls, tool invocations, and their triggered file and network effects. Each prompt opens a span. LLM calls, tool calls, and system effects within that span inherit its semantic label, connecting intent to system behavior without runtime instrumentation. Public benchmark datasets enter as operation JSONL with mapping rules deriving fields before stack construction. Standard Chrome Trace Event format can also

be imported, with non-AGENTPPROF protocol fields preserved as operation fields.

Output formats include pprof protobuf (compatible with `go tool pprof -http`), folded-stack text (compatible with `flamegraph.pl`), standalone SVG flame graphs, and JSON summaries. All outputs are deterministic given the same input and profile spec.

AGENTPPROF is the offline profiling component of the AgentSight observability framework. AgentSight provides real-time eBPF-based capture of SSL traffic, process events, and file/network effects. AGENTPPROF aggregates the recorded traces into semantic profiles. The two are complementary: AgentSight answers “what happened during this run” (debugging), AGENTPPROF answers “where did the budget go across runs” (profiling).

## 5 Evaluation

We organize the evaluation around three research questions.

*Evaluation setup.* We use two classes of data. *Real agent sessions:* 325 readable local sessions (198 Codex, 50 Claude, 77 Claude sub-agent) from a multi-month development project, containing 183,714 system observations. *Public labeled traces:* 15 families covering web, mobile, desktop, software, API, and dialogue agents (Table 2), totaling 47,590 operations through mapping rules. All public traces are real agent or human executions collected and labeled by their respective benchmark projects. Four oracle-rich families (AgentRewardBench, SATraj-OS, AgentNet, OSWorld-Human) provide hidden ground-truth labels for RQ2.

### 5.1 RQ1: Does Semantic Profiling Separate What Flat Views Mix?

The strongest argument against semantic profiling is that traditional tools “can already see the same information.” RQ1 tests this claim directly by measuring how much system-effect weight falls into *mixed-intent* buckets—buckets containing observations from multiple semantic regions—under different projection choices.

Table 3 shows the result on 325 real Codex/Claude sessions. Without semantic labels, 90.4% of system-effect weight falls into mixed buckets: a flat cargo test counter shows 2,903 executions but cannot distinguish whether they came from review, debug, refactor, or test prompts. Adding only the prompt label reduces mixed weight to 36.7% and residual to 7.5%. The session label alone barely helps (84.4%), because sessions typically contain multiple intent phases.

A concrete example: `process:cargo; effect:test; status:ok` appears under 48 distinct prompt tags. The top tag (refactor) accounts for only 32.0%; the remaining 68.0% comes from review, research, test, explain, and others. A flat summary or process audit merges all of these.

As a robustness check, a label-permutation test (1,000 shuffles within each session) confirms the separation is not random ( $p = 0.001$ ). The primary evidence is the mixing reduction itself. The permutation test verifies statistical significance. RQ3 evaluates label quality from a complementary angle: agreement with ground-truth annotations on public datasets.

RQ1 also tests whether query-time stack selection (R2) adds value beyond flat label grouping. The same two-abstraction model covers all 15 public trace families (Table 2) without type-specific

**Table 2: Public labeled trace families. All traces are real agent or human executions. The bottom four rows provide oracle labels for RQ2.**

| Family                                | Native labels                 | Access         | Trace origin           | Eval. role |
|---------------------------------------|-------------------------------|----------------|------------------------|------------|
| WebLINX, Mind2Web, AgentTrek, WebShop | Web actions, tasks, rewards   | HF / public    | Real web agents        | Coverage   |
| API-Bank, ToolBench, tau-bench        | API calls, outcomes           | HF / JSONL     | Real API agents        | Coverage   |
| SWE-agent                             | Issue trajectories            | HF viewer      | Real coding agent      | Coverage   |
| AndroidCtrl, GUI-Odyssey, ScaleCUA    | Mobile/GUI actions            | Public/HF      | Real mobile agents     | Coverage   |
| AgentRewardBench                      | Success, side-effect, looping | HF annotations | Real BrowserGym agents | RQ2 tasks  |
| SATraj-OS                             | Safety, reward, attack type   | HF config      | Real desktop agents    | RQ2 task   |
| OSWorld-Human                         | Human grouped-action traces   | GitHub JSON    | Real human OS use      | RQ2 task   |
| AgentNet                              | Step correctness, redundancy  | HF JSONL       | Real desktop agents    | RQ2 tasks  |

**Table 3: Semantic-axis ablation on 183,714 system observations from 325 real sessions. All rows preserve the same total weight.**

| Projection         | Unique stacks | Mixed wt.% | Residual% |
|--------------------|---------------|------------|-----------|
| No semantic labels | 11,967        | 90.4       | 44.7      |
| Session label only | 15,027        | 84.4       | 33.4      |
| Prompt label only  | 24,703        | 36.7       | 7.5       |
| Session + prompt   | 26,829        | 0.0        | 0.0       |

objects. On the four oracle-rich families, the same 13,265 operations fold from 9 dataset stacks to 57 phase stacks, 226 semantic stacks, 455 action stacks, or 3,757 fixed-session stacks by changing only the stack fields. This is not achievable with a flat GROUP BY on a single label: different stack depths answer different questions (task-level budget vs. action-level duplication vs. session-level drilldown), and the profiler produces all of them from the same operations.

The multi-view design is further validated by metric divergence. The time and tokens views of the same sessions share only 7/10 top prompt categories (Spearman  $\rho = 0.623$ ). network-tagged prompts rank 8th by time (I/O wait) but 93rd by tokens.

These quantitative results translate to concrete aggregate insights. The tokens view (Figure 2) reveals that review-tagged prompts dominate the model budget, followed by git, code, edit, and debug—a distribution that a timeline of 80,000 individual LLM calls cannot produce without external aggregation. The files view shows debug-tagged prompts across different sessions repeatedly accessing the same source files, aggregated into a single wide bar that a timeline shows as isolated events. The network view attributes domain contacts to prompt categories, and the files view identifies accesses outside the project root, supporting security auditing.

## 5.2 RQ2: Can Profiler Groups Localize Hidden Labeled Problems?

RQ2 asks whether the profiler’s top groups correspond to real labeled problems—whether categories that concentrate failures, safety violations, or wasted effort surface at the top of ranked output. The methodology is analogous to injecting known bottlenecks in a CPU

[Flame-graph screenshots: tokens view (top) and files view (bottom) showing budget distribution and file-access patterns across sessions]

**Figure 2: Real flame graphs from a development project. Top: tokens view (width = token count). Bottom: files view (width = file-operation count).**

profiler evaluation: public benchmark datasets provide ground-truth labels (failure, safety violation, redundant step, human-task boundary); we hide these labels, run the profiler, rank the resulting groups, and check whether the top groups contain the hidden positives.

*Benchmark setup.* Six tasks across four datasets: AgentReward-Bench looping and side-effect, SATraj-OS unsafe operation, AgentNet incorrect and redundant step, and OSWorld-Human group start. Together: 34,539 operations, 3,699 positives. Six views (flat, fixed-session, dataset-native, raw-action, operation-stack, label-drilldown) and four rankers (width, visible-risk, query-aware, oracle).

*Localization results.* Table 4 summarizes the results. Flat summaries recover all positives but require inspecting everything (Work@5 = 1.0). Fixed-session drilldown finds a first positive quickly (WTFP = 0.004) but fragments the workload into 285 groups with only 2.3% top-five recall. Operation-stack query-aware profiling inspects 9.4% of the work in the top five groups, achieves 39.0% budget-30 recall, and reduces groups to 157.5.

The result is a Pareto tradeoff, not dominance. The comparison also serves as evidence for the two-abstraction model (C1): fixed-session drilldown and dataset-native hierarchy are alternative abstractions that fix the hierarchy at session or dataset boundaries. Operation stacks subsume both as special cases (include session in the stack to recover fixed-session; include dataset to recover dataset-native). On localization, operation stacks improve top-five recall over fixed-session on 5/6 tasks, reduce median group count by 45%, and save 90% of inspection work versus flat summaries. Fixed-session drilldown retains lower work-to-first-positive on 4/6 tasks: profilers and debuggers are complementary.

**Table 4: RQ2 hidden-label localization (medians across six tasks). Hidden policies serve only as upper bounds. WTFP = work-to-first-positive. “—” = group count varies by task; no single median is reported.**

| Policy                     | Hidden | AP    | R@5   | F1@5  | Work@5 | R@30% | WTFP  | Groups |
|----------------------------|--------|-------|-------|-------|--------|-------|-------|--------|
| flat:width                 | 0      | 0.168 | 1.000 | 0.287 | 1.000  | 0.000 | 1.000 | 1.0    |
| fixed-session:query-aware  | 0      | 0.348 | 0.023 | 0.043 | 0.016  | 0.356 | 0.004 | 285.0  |
| dataset-native:query-aware | 0      | 0.357 | 0.867 | 0.282 | 0.854  | 0.338 | 0.058 | —      |
| raw-action:query-aware     | 0      | 0.274 | 0.244 | 0.220 | 0.151  | 0.333 | 0.037 | —      |
| op-stack:query-aware       | 0      | 0.312 | 0.188 | 0.198 | 0.094  | 0.390 | 0.038 | 157.5  |
| op-stack:oracle-upper      | 1      | 0.599 | 0.300 | 0.403 | 0.044  | 0.564 | 0.012 | —      |
| label-drilldown:oracle     | 1      | 1.000 | 0.670 | 0.797 | 0.115  | 1.000 | 0.038 | —      |

Statistical validation supports these comparisons: bootstrap (10,000 resamples) and label-permutation (2,000 trials) confirm stable AP deltas exceeding the 95% prevalence/group-size null on all six tasks.

*Actionability: which configuration knobs explain the results?* Executable profile-spec patches modify stack fields, mapping rules, or rankers, then re-run over the same visible operations. Five of six patches improve AP, top-five lift, and first-positive work (median AP delta 0.038, median top-five lift delta 0.575). Three reusable patterns emerge: (1) safety/side-effect tasks benefit from field and ranker changes; (2) step-quality tasks benefit from operation-stack localization followed by fixed-session drilldown for examples; (3) human-boundary tasks require boundary-derived fields. The rejected OSWorld-Human patch shows that visible rank features alone are insufficient for boundary objectives; folding held-out boundary-derived fields improves AP from 0.240 to 0.258 and reduces groups from 108 to 74, with an explicit work tradeoff.

Rank-feature ablations identify 7 critical feature rows (repeat\_signal, write-action, status=success) and 3 misleading rows. Guided repairs improve AP on 2/3 misleading cases but use offline scored feedback, not an automatic learned ranker.

### 5.3 RQ3: How Reliable Is Intent Recognition?

RQ1 and RQ2 assume that semantic labels are useful. RQ3 tests the label-derivation pipeline itself. For structured traces (public datasets), we evaluate the *mapping* path against ground-truth annotations. For free-form traces (real sessions), we evaluate the *intent-recognition* path at the scale and syntax level; quality assessment of the LLM/regex tagger against free-form ground truth remains future work.

*Scale and syntax validity.* On 325 real sessions, the LLM tagger processes 118,021 tag requests (2,859 prompt rows, 114,837 LLM-call events). Of these, 82,886 hit the label cache; 35,136 are real llama.cpp calls to a local 3B-parameter model. Final tag failures: 0. On a fixed set of 300 redacted fragments, three model sizes (0.6B, 1.1B, 3B) all produce 900/900 syntactically valid outputs; p95 latencies are 23, 18, and 31 ms respectively.

*Ground-truth agreement on public datasets.* Public benchmark datasets provide native action labels that serve as ground truth for mapping quality. We run a leave-dataset-out evaluation: for each of 9 datasets, we train mapping rules on the other 8 and measure V-measure between the mapping-derived phase field and the dataset’s

**Table 5: Leave-dataset-out mapping quality. V-measure between mapping-derived phase and native action labels. 13,265 total operations across 9 held-out evaluations.**

| Held-out dataset | Ops   | V-measure | Boundary F1 |
|------------------|-------|-----------|-------------|
| mind2web         | 774   | 1.000     | 1.000       |
| webshop-expert   | 2,504 | 1.000     | 1.000       |
| swe-agent        | 496   | 0.926     | 0.962       |
| weblinx-chat     | 500   | 0.872     | 0.860       |
| agenttrek        | 200   | 0.862     | —           |
| gui-odyssey      | 7,868 | 0.811     | 0.842       |
| android-control  | 9     | 0.716     | 0.727       |
| toolbench        | 866   | 0.134     | 0.353       |
| api-bank         | 48    | 0.000     | —           |

native action labels, plus boundary F1 between mapped phase boundaries and native action boundaries.

Table 5 shows the results. On 7/9 datasets, V-measure exceeds 0.7, indicating strong agreement between mapping-derived and native labels. The two low-scoring datasets (toolbench, api-bank) have atypical action vocabularies that the held-out mapping rules do not cover. Mapping reduces unique stacks on 6/9 datasets with zero negative regressions.

A supervised boundary-derivation backend further validates the field pipeline. On OSWorld-Human held-out adjacent pairs, a learned boundary predictor achieves F1 0.77, beating the best simple baseline (always\_boundary, F1 0.71). Learned backends beat simple baselines on 4/5 tested boundary families, while AgentRewardBench looping is fully explained by repeat\_signal\_change (baseline F1 1.0). This supports field derivation as a guarded configuration step, not automatic intent discovery.

Two mapping-rule variants (differing in API/tool phase precedence) run on the same 13,265 operations and produce identical V-measure on 7/9 datasets, with the two divergent datasets differing by  $\leq 0.17$ . Mapping quality is robust to minor rule variations on well-covered action vocabularies. Direct cross-backend comparison between the regex and LLM taggers on the same free-form prompts remains future work.

Syntactic validity does not imply semantic quality. TinyLlama 1.1B achieves high exact-stability (279/300 identical across repeats) but concentrates on a narrow label set. The 3B model produces a broader vocabulary (328 unique prompt tags) but includes noisy entries. A display-governance pipeline reduces unique tags from

1,546 to 1,364 through deterministic aliases, raising top-20 support coverage from 93.7% to 95.2% without overwriting raw tags. This pipeline manages noise but does not establish semantic adequacy—that is what the ground-truth and cross-backend checks above provide.

All results are backed by profile specs for deterministic replay: 76 tracked specs re-executed twice (152 invocations) produce identical output, with median runtime 1.6 s per spec.

## 6 Related Work

**Profiling and flame graphs.** Flame graphs [1] and pprof [2] establish the folded-stack lineage: profiles are weighted samples with location hierarchies. Pprof supports sample labels and can visualize them as pseudo stack frames through tag-root/tag-leaf options. Perfetto generalizes trace ingestion with SQL-based analysis and derived events [3]. These systems support query-time aggregation, so AGENTPPROF does not claim novelty for flexible aggregation alone. The difference is the domain: samples are agent operations, frames are semantic labels derived from natural-language prompts through intent recognition, and the evaluation scores profiler output against hidden agent-trajectory labels.

**Agent tracing and LLM observability.** OpenTelemetry GenAI and OpenInference standardize GenAI spans [4, 5]. LangSmith [6], Langfuse [7], and Phoenix [8] provide trace visualization, session management, evaluations, and dashboards. AgentOps [9] argues for structured agent observability covering goals, plans, tools, and artifacts. These systems excel at single-execution debugging. AGENTPPROF complements them with cross-session aggregate profiling. Datadog LLM Observability and Laminar Signals take steps toward semantic aggregation by clustering user messages or extracting structured events, but they characterize *input distributions*, not *resource attribution*.

**Agent failure diagnosis.** AgentRx [10] releases annotated failure trajectories and uses constraint checking plus an LLM judge for root-cause localization. TELBench [11] builds span-localization benchmarks from semantic error annotations. AgentAtlas argues that outcome leaderboards miss trajectory-level quality. These systems perform *span-level* fault localization within a single execution: which step failed and why. AGENTPPROF performs *category-level* aggregate profiling: which categories of work produce the most failures, consume the most budget, or generate repeated system effects across sessions.

## 7 Conclusion

Agent observability needs profiling, not only debugging. The obstacle is that natural-language prompts lack the deterministic identifiers that make flame-graph folding work. AGENTPPROF shows that two abstractions—operations and operation stacks—close this gap once intent recognition restores deterministic labels. The evaluation confirms that semantic profiling separates information flat views merge, localizes labeled problems with less work than both flat summaries and fixed-session drilldown, and produces labels that agree with ground-truth annotations across heterogeneous public datasets.

Profilers and debuggers are complementary: fixed-session drill-down finds a first positive faster on 4/6 tasks (RQ2), while the

profiler identifies which categories deserve attention. AGENTPPROF supports both by including or excluding session fields in the stack query. The profiler’s structured output also enables automated consumption by analysis agents, positioning semantic profiling as infrastructure for agent self-diagnosis.

The main limitation is single-project real-session data; multi-project evaluation and trace-ecosystem integration (importing OpenTelemetry and LangSmith traces as operations) are the most impactful next steps.

## References

- [1] Brendan Gregg. Flame Graphs. <https://www.brendangregg.com/flamegraphs.html>.
- [2] Google pprof. <https://github.com/google/pprof>.
- [3] Perfetto Trace Processor. <https://perfetto.dev/docs/analysis/trace-processor>.
- [4] OpenTelemetry GenAI Semantic Conventions. <https://github.com/open-telemetry/semantic-conventions-genai>.
- [5] OpenInference Specification. <https://arize-ai.github.io/openinference/spec/>.
- [6] LangSmith Observability. <https://docs.langchain.com/langsmith/observability>.
- [7] Langfuse Observability. <https://langfuse.com/docs/observability/overview>.
- [8] Arize Phoenix. <https://arize.com/docs/phoenix>.
- [9] Liming Dong et al. AgentOps: Enabling Observability of LLM Agents. arXiv:2411.05285, 2024.
- [10] Shraddha Barke et al. AgentRx: Diagnosing AI Agent Failures. arXiv:2602.02475, 2025.
- [11] NJU-LINK. Where Do Deep-Research Agents Go Wrong? arXiv:2606.02060, 2026.
- [12] Xing Han Lù et al. AgentRewardBench. arXiv:2504.08942, 2025.
- [13] Tianbao Xie et al. OSWorld. arXiv:2404.07972, 2024.
- [14] Xinyuan Wang et al. OpenCUA. arXiv:2508.09123, 2025.
- [15] Shanghai AI Lab. Safactory. arXiv:2605.06230, 2026.
- [16] WukLab OSWorld-Human. <https://github.com/WukLab/osworld-human>.
- [17] xlangai AgentNet. <https://huggingface.co/datasets/xlangai/AgentNet>.